

TP9 : parcours de graphes

ZFAI

INTRODUCTION

Pour simplifier la manipulation et l’affichage des graphes, on fournit pour ce TP un squelette du code des différents exercices. Concernant l’affichage, il est possible d’effectuer un affichage simple sous forme de liste d’adjacence ou sous forme de matrice d’adjacence. Dans le cas où les bibliothèques `matplotlib` et `networkx` sont installées, il est possible d’avoir un affichage plus visuel, mais ceci n’est absolument pas nécessaire pour la compréhension du TP.

À FAIRE

Exercice 1 (Représentation des graphes). Pour représenter les graphes en Python, on choisit de créer une classe munie des méthodes qu’il faudra implémenter. Dans notre cas, un graphe `G` possède deux attributs :

- `n` qui correspond au nombre de sommets du graphe,
- `adj` qui correspond à la liste d’adjacence.

Si `G` est un graphe à `n` sommets, pour simplifier la représentation l’ensemble des sommets de `G` est alors $\llbracket 0, n-1 \rrbracket$. Pour un tel graphe, `G.n` est alors le nombre `n`, `G.adj` est la liste d’adjacence de `G`. Un exemple est présent dans le fichier.

Dans une telle représentation, les graphes considérés sont orientés. On rappelle que dans ce cas, les graphes non orientés deviennent des cas particuliers de graphes orientés : ce sont les graphes $G = (S, A)$ qui vérifient la propriété suivante :

$$\forall (u, v) \in S \times S, (u, v) \in A \iff (v, u) \in A.$$

Autrement dit, ce sont des graphes où l’ensemble des arcs est symétrique.

Compléter la fonction `est_non_oriente` qui renvoie `True` si le graphe correspondant est non orienté, et `False` sinon.

Exercice 2 (Parcours simple à partir d’un sommet). 1. La méthode `affichage_profondeur` présente dans le fichier `TP9.py` permet d’afficher les sommets lors d’un parcours en profondeur partant d’un sommet `s`. Compléter la méthode `parcours_profondeur` pour que la fonction renvoie la liste des sommets dans l’ordre de l’affichage de `affichage_profondeur`.

2. La méthode `affichage_largeur` présente dans le fichier `TP9.py` permet d’afficher les sommets lors d’un parcours en largeur partant d’un sommet `s`. Compléter la méthode `parcours_largeur` pour que la fonction renvoie la liste des sommets dans l’ordre de l’affichage de `affichage_largeur`.

3. La méthode `affichage_profondeur_rec` présente dans le fichier `TP9.py` permet d’afficher les sommets lors d’un parcours en profondeur implémenté en récursif partant d’un sommet `s`.

Compléter la méthode `parcours_profondeur_rec` pour que la fonction renvoie la liste des sommets dans l’ordre de l’affichage de `affichage_profondeur_rec`.

4. Vérifier vos fonctions et comparer avec les valeurs tests.

5. (★) Expliquer brièvement la différence entre les deux parcours en profondeur.

Exercice 3 (Connexité). On dit qu’un graphe $G = (S, A)$ non orienté est connexe si pour tout couple de sommets (s, t) il existe un chemin commençant par `s` et aboutissant à `t`. Pour montrer qu’un graphe est connexe, on peut effectuer un parcours dans le graphe en commençant par un sommet quelconque et constater que l’on finit par voir tous les sommets du graphe.

1. Compléter la méthode `est_connexe_par_largeur` qui renvoie `True` si le graphe est connexe et `False` sinon. On utilisera un parcours en largeur et on veillera à vérifier que le graphe est bien non orienté.

2. Compléter la méthode `est_connexe_par_profondeur` qui renvoie `True` si le graphe est connexe et `False` sinon. On utilisera un parcours en profondeur et on veillera à vérifier que le graphe est bien non orienté.

Exercice 4 (Distance à un sommet). Étant donné un graphe fini $G = (S, A)$, deux sommets `s` et `t` de `G`, la distance de `s` à `t` désigne la plus petite longueur possible d’un chemin qui commence par `s` et qui finit par `t`. Dans le cas où un tel chemin n’existe pas, on convient que la distance est alors $+\infty$.

1. En adaptant le parcours en largeur, compléter la méthode `distance` qui renvoie la liste des distances du sommet `s` aux autres sommets.

- En adaptant la méthode `distance`, compléter la méthode `parents_largeur` qui renvoie une liste L vérifiant la propriété suivante : si s est le sommet initial du parcours, $L[s] = s$. Si v est à distance k de s , alors $L[v]$ est à distance $k - 1$ de s et $(L[v], v)$ est un arc du graphe.
- À l'aide de la méthode `parents_largeur`, compléter la méthode `chemins_optimaux` qui renvoie la liste des chemins optimaux partant du sommet s .

Exercice 5 (Détection de cycle). On rappelle que dans un graphe orienté $G = (S, A)$, un cycle est un chemin non vide $s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_n$ avec $n \geq 1$ et $s_0 = s_n$. On considère la variante suivante du parcours en profondeur :

```
def ParcoursProfondeur(G) :
    /* G correspond au graphe */
    Initialiser un tableau  $t$  à 0 mémorisant l'état des sommets
    /* 0 pour jamais vus, 1 pour traitement en cours : le sommet est encore dans la pile de récursion,
       2 pour le traitement du sommet est fini ; */
    def ParcoursProfondeurAux(G,s) :
        si  $t[s] = 0$  alors
            /*  $t[s] = 0$  signifie que le parcours en profondeur commençant par  $s$  n'a jamais eu lieu */
             $t[s] \leftarrow 1$ ;
            pour  $v$  voisin cible de  $s$  faire
                | ParcoursProfondeurAux (G,v)
            fin
             $t[s] \leftarrow 2$ ; /*  $t[s]=2$  signifie que le parcours en profondeur commençant par  $s$  est fini */
        fin
        si  $t[s] = 1$  alors
            /*  $t[s] = 1$  signifie que le parcours en profondeur commençant par  $v$  est en cours */
            Afficher  $s$  est encore dans la pile
        fin
    fin
    pour  $s$  sommet de  $G$  faire
        | ParcoursProfondeurAux (G,s)
    fin
fin
```

- (*) Montrer que G contient un cycle si et seulement si, lors d'un parcours en profondeur donné par la variante, le message `est encore dans la pile` est affiché.
- En déduire la méthode `contient_cycle` qui renvoie `True` si le graphe contient un cycle et `False` sinon. On pourra adapter l'algorithme de parcours en profondeur présenté.

POUR LES PLUS RAPIDES

Exercice 6. Étant donné un graphe non orienté $G = (S, A)$, un sommet s , la composante connexe de G contenant s est l'ensemble des sommets atteignables par s à l'aide de chemins dans G .

Compléter la méthode `composantes_connexe`. Elle renvoie la liste des composantes connexes de G . On pourra adapter un parcours en largeur.

Exercice 7. Les graphes peuvent être utilisés pour modéliser des relations d'ordre dans un ensemble fini. En effet, si on considère une relation d'ordre sur un ensemble fini S , on peut construire le graphe correspondant en considérant qu'il existe une arête de s_1 vers s_2 si et seulement si $s_1 > s_2$ et qu'il y a aucun élément s vérifiant $s_1 > s > s_2$. On pourra alors remarquer que $t \leq s$ si et seulement si il existe un chemin de s vers t dans le graphe construit. Un tri topologique du graphe est alors un ordre total $\leq_T$ sur les sommets qui est compatible avec l'ordre du graphe : pour tout sommet s, t on a $s \leq t \implies s \leq_T t$.

- Montrer qu'un graphe G admet un tri topologique si et seulement si G ne contient pas de cycle.
- Compléter la méthode `tri_topo` qui renvoie un tri topologique dans le cas où celui-ci existe. Dans le cas contraire, on lève une exception signalant la présence de cycle.